Datenexfiltration über ICMP

Dokumentation zur Prüfungsleistung Network Security - TINF24CS2

Devi Müller

---

1. Worum es geht

Die Aufgabe war, einen Datenabfluss über das Netzwerk nachzustellen: Ein Programm A liest eine Textdatei aus (die Paragrafen 202c und 202d StGB) und schickt deren Inhalt an ein Programm B, das die Daten wieder zusammensetzt, auf dem Bildschirm anzeigt und abspeichert. Beide Programme schneiden ihren eigenen Netzwerkverkehr als pcap mit.

Als Transportweg wurde **ICMP** gewählt; beide Programme sind in Python mit der Bibliothek Scapy umgesetzt. Die eigentlichen Nutzdaten wandern nicht im Klartext über die Leitung, sondern Base64-codiert in der Nutzlast von ganz normalen Ping-Paketen. Dieses Dokument erklärt, warum ICMP gewählt wurde, wie der Transport funktioniert, wie codiert und decodiert wird und wie sichergestellt wird, dass am Ende Zeichen für Zeichen dasselbe ankommt, was losgeschickt wurde.

Die Vorlage ist rund 1,6 Kilobyte groß. Nach der Base64-Codierung ergibt sich daraus ein Strom von knapp 2,2 Kilobyte, der in Stücke von höchstens 36 Byte geteilt wird. Das ergibt 61 Datenpakete, dazu je ein Paket für den Anfang und das Ende der Übertragung. Dieser Umfang genügt, um alle relevanten Mechanismen an einem konkreten Beispiel zu zeigen: Transport, Codierung, Prüfsummen und automatische Fehlerkorrektur.

2. Warum ICMP

Die Wahl von ICMP beruht auf mehreren Gründen. Das Protokoll ist in praktisch jedem Netz zulässig, da die meisten Firewalls Echo-Requests und -Replies passieren lassen und Ping-Verkehr üblicherweise nicht als Angriff eingestuft wird. Dadurch eignet sich ICMP für eine verdeckte Übertragung, da sie sich auf Protokollebene nicht von gewöhnlichem Ping-Verkehr unterscheidet.

Ein weiterer Grund ist technischer Natur: Ein ICMP-Echo-Request kann eine frei wählbare Nutzlast tragen, und der Empfänger antwortet protokollgemäß mit einem Echo-Reply. Damit stellt das Protokoll zwei benötigte Elemente bereit: einen Platz für die zu übertragenden Daten (die Payload) sowie einen Rückkanal für Bestätigungen über den ohnehin vorgesehenen Reply. Der Ablauf aus Request und Reply entspricht damit genau der von ICMP vorgesehenen Nutzung; lediglich die Nutzlast wird zweckentfremdet, während das Protokoll selbst korrekt eingehalten wird.

3. Wie die Daten übers Netz gehen

Programm A verschickt die Daten in vielen kleinen ICMP-Echo-Requests (Typ 8). In jede Nutzlast wird ein eigener kleiner Rahmen (ein „Frame“) gelegt, der aus einem Kopf und dem eigentlichen Datenstück besteht. Programm B hört mit einem Raw-Socket auf eingehende Echo-Requests, erkennt an einer Kennung, welche davon aus Programm A stammen, und schickt für jedes korrekt empfangene Paket einen Echo-Reply (Typ 0) als Bestätigung zurück. Fremder Ping-Verkehr wird dabei ignoriert.

Dabei wird der von ICMP vorgesehene Ablauf bewusst eingehalten, statt ihn zu brechen: Jeder Request bekommt eine feste Identifier-Nummer und eine fortlaufende Sequenznummer im ICMP-Kopf, genau wie ein echter Ping sie setzt, und B antwortet darauf mit einem passenden Echo-Reply. Von außen betrachtet ist das also ein formal korrekter Ping-Austausch, nur dass in der Nutzlast nicht die üblichen Füllbytes stehen, sondern der codierte Rahmen. Der Verkehr bleibt dadurch

protokollkonform und unterscheidet sich auf Protokollebene nicht von einem gewöhnlichen Ping-Austausch.

Damit der Verkehr unauffällig bleibt, ist die Paketgröße bewusst an einen normalen Ping angelehnt. Ein Standard-Ping unter macOS oder Linux trägt 56 Byte Nutzlast; jedes Programm verwendet daher ebenfalls immer genau 56 Byte ICMP-Payload. Davon entfallen 20 Byte auf den Rahmenkopf und bis zu 36 Byte auf Daten; kürzere INIT-, FIN- und ACK-Frames werden mit Nullbytes aufgefüllt. Anhand der Paketgröße allein lässt sich der Datenverkehr dadurch nicht von regulärem Ping-Verkehr unterscheiden.

Der Rahmenkopf ist 20 Byte lang und immer gleich aufgebaut:

Feld	**Größe**	**Bedeutung**
magic	4 B	Kennung „DHBW“, um eigene Pakete von fremdem Ping zu trennen
version	1 B	Protokollversion (aktuell 1)
ptype	1 B	Pakettyp: INIT, DATA, FIN oder ACK
seq	4 B	laufende Nummer des Frames
total	4 B	Gesamtzahl der Datenframes
data$_{len}$	2 B	Länge des Datenteils
crc32	4 B	Prüfsumme über den Datenteil
data	0–36 B	das eigentliche (codierte) Datenstück

Tabelle 1: Aufbau eines Frames innerhalb der ICMP-Nutzlast.

Eine Übertragung läuft in drei Phasen ab. Zuerst schickt A ein **INIT**-Paket mit dem SHA-256-Fingerabdruck der Originaldatei und sagt damit auch, wie viele Datenpakete folgen werden. Danach kommen die **DATA**-Pakete, jedes mit einem Stück der codierten Daten und seiner eigenen laufenden Nummer. Zum Schluss markiert ein **FIN**-Paket das Ende, woraufhin B alles zusammensetzt und prüft. Die **ACK**-Pakete sind die Bestätigungen, die B über den Echo-Reply zurückschickt.

Wichtig ist, dass die Daten tatsächlich durch den Netzwerkstack des Betriebssystems gehen und nicht etwa über einen direkten Funktionsaufruf zwischen den Prozessen. Scapy baut das IP/ICMP-Paket und übergibt es an einen Socket; der Kernel verschickt es, und auf der Gegenseite liest ein Raw-Socket es wieder ein. In Wireshark ist jeder Schritt sichtbar.

Für ein einzelnes Datenpaket sieht der Ablauf zwischen A und B immer gleich aus:

1. A baut den Frame zusammen: Kopf mit laufender Nummer und Prüfsumme, dahinter das bis zu 36 Byte große Datenstück.
2. A verpackt den Frame in einen ICMP-Echo-Request und schickt ihn an B. Parallel notiert A sich, dass es für diese Nummer noch auf eine Bestätigung wartet.
3. B empfängt den Request, erkennt an der Kennung das eigene Format und rechnet die Prüfsumme nach.
4. Stimmt sie, legt B das Datenstück unter seiner Nummer ab und schickt einen Echo-Reply mit der Bestätigung (ACK) zurück. Stimmt sie nicht, schickt B stattdessen ein NACK.
5. A bekommt die Antwort, hakt das Paket ab und geht zum nächsten über. Bleibt die Antwort aus oder kommt ein NACK, sendet A dasselbe Paket erneut.

Dieser kurze Hin-und-Her-Takt wiederholt sich für jedes der 61 Datenpakete, eingerahmt vom INIT am Anfang und vom FIN am Ende.

4. Codierung und Decodierung

Verschlüsselt wird laut Aufgabenstellung nichts. Gefordert ist ein Codierungsverfahren, und die Wahl fiel auf **Base64**. Der Grund ist praktischer Natur: Der Originaltext enthält Umlaute, Paragrafenzeichen und Zeilenumbrüche, also Bytes, die sich nicht ohne Weiteres unverändert durch eine Paketnutzlast übertragen lassen, ohne dass es zu Problemen mit Sonderzeichen kommt. Base64 bildet beliebige Bytes auf einen eingeschränkten, unkritischen Zeichenvorrat (A–Z, a–z, 0–9, `+`, `/`) ab und erzeugt daraus einen gleichförmigen Strom, der sich in gleich große Stücke zerlegen lässt. Der Inhalt ist dadurch zugleich nicht mehr im Klartext lesbar; ein Mitschnitt zeigt zunächst nur eine Zeichenfolge ohne erkennbare Struktur.

Das folgende Beispiel veranschaulicht dies: Aus dem Wort `§ 202c` (das wegen des Paragrafenzeichens mehrere Bytes umfasst) wird nach der Codierung die Zeichenfolge `wqcgMjAyYw==`. Solche Zeichenketten reihen sich in den Datenpaketen aneinander, ohne dass der zugrunde liegende Gesetzestext auf den ersten Blick erkennbar ist.

Der Ablauf gestaltet sich wie folgt: Programm A liest die Datei als Bytes ein, verarbeitet sie mit `base64.b64encode` und zerlegt das Ergebnis in Stücke von höchstens 36 Byte. Jedes Stück wandert in ein DATA-Paket. Programm B sammelt die Stücke, sortiert sie nach ihrer laufenden Nummer, hängt sie wieder aneinander und ruft `base64.b64decode` auf. Heraus kommen exakt die ursprünglichen Bytes, die dann als Text angezeigt und gespeichert werden. Weil Base64 eine umkehrbare Eins-zu-eins-Abbildung ist, geht bei Codierung und Decodierung grundsätzlich kein Zeichen verloren. Alle Verluste, die abgefangen werden müssen, passieren auf dem Transportweg, nicht bei der Codierung.

Die korrekte Übertragung von Umlauten und Sonderzeichen wird dadurch sichergestellt, dass die Datei durchgehend als Bytes behandelt und erst bei der Anzeige am Ende wieder als UTF-8-Text interpretiert wird. Dadurch entsteht an keiner Stelle der Übertragung eine Mehrdeutigkeit bei der Zeichensatz-Interpretation.

5. Integrität und Fehlerkorrektur

Dass alles ankommt, bleibt nicht dem Zufall überlassen. Geprüft wird auf zwei Ebenen, und bei Fehlern wird aktiv nachkorrigiert.

Pro Paket trägt jeder Frame eine CRC32-Prüfsumme über seinen Datenteil im Kopf. Programm B rechnet die Prüfsumme beim Empfang neu und vergleicht. Stimmt sie, gilt das Paket als sauber; stimmt sie nicht, war das Paket unterwegs beschädigt und wird verworfen. CRC32 bietet keinen kryptografischen Schutz, eignet sich jedoch zur zuverlässigen Erkennung von Bitfehlern bei der Übertragung. Schon ein einziges gekipptes Bit im Datenteil führt zu einer völlig anderen Prüfsumme. B merkt den Fehler also sofort, noch bevor das kaputte Stück überhaupt in den Gesamtpuffer wandert. Genau dieses Verhalten wurde auch gezielt getestet: Dreht man in einem Frame ein Bit um, schlägt die Prüfung erwartungsgemäß an und das Paket wird als fehlerhaft markiert.

Für die Gesamtdatei kommt zusätzlich SHA-256 ins Spiel. Den Fingerabdruck der Originaldatei schickt A gleich zu Beginn im INIT-Paket mit. Wenn B am Ende alles zusammengesetzt und decodiert hat, bildet es seinerseits den SHA-256 über das Ergebnis und vergleicht. Nur wenn beide Werte identisch sind, ist garantiert, dass wirklich die komplette Datei fehlerfrei angekommen ist und nicht etwa ein Stück vertauscht oder verloren wurde.

Die eigentliche **Fehlerkorrektur** läuft über ein einfaches, aber wirksames Verfahren: Stop-and-Wait mit Bestätigungen, in der Netzwerktechnik als ARQ bekannt. A schickt ein Paket und wartet, bis B es bestätigt. Dabei gibt es zwei Fälle. Kam das Paket sauber an, schickt B ein ACK mit dem Status „OK“, und A geht zum nächsten Paket über. War die Prüfsumme falsch, schickt B ein NACK zurück, und A sendet dasselbe Paket sofort noch einmal. Bleibt die Bestätigung ganz aus (etwa weil

ein Paket komplett verloren ging), läuft bei A nach zwei Sekunden eine Zeitschranke ab, und auch dann wird erneut gesendet. Bis zu fünf Versuche pro Paket sind vorgesehen; erst danach gilt die Übertragung als gescheitert. Ein Datenpaket, das trotz vorheriger Bestätigung noch einmal eintrifft (etwa weil das ACK selbst verloren ging), legt B einfach erneut unter derselben Sequenznummer ab: Das Ergebnis ändert sich dadurch nicht, und B bestätigt wie gewohnt.

Zusammengenommen greifen die drei Mechanismen ineinander: CRC32 erkennt kaputte Pakete, das ARQ-Verfahren sorgt dafür, dass fehlende oder beschädigte Pakete automatisch nachgeliefert werden, und SHA-256 ist am Ende der Beweis, dass das Ergebnis vollständig und korrekt ist.

6. Mitschnitt und Nachvollziehbarkeit

Beide Programme erzeugen während der Laufzeit selbst eine pcap-Datei. Dazu läuft in jedem Programm ein Scapy-Sniffer mit, der jedes ICMP-Paket (in beide Richtungen und ungefiltert) sofort in die Datei schreibt. So entstehen zwei Mitschnitte, die man hinterher in Wireshark Paket für Paket nachvollziehen kann: die Echo-Requests mit den Daten und die Echo-Replies mit den Bestätigungen.

Zusätzlich schreibt jedes Programm ein Logbuch, das parallel auf der Konsole und in einer Datei erscheint. Dort steht im Klartext, welches Paket gerade raus- oder reingeht, mit welcher Sequenznummer und beim wievielten Versuch, sowie ob eine ACK- oder NACK-Antwort kam. Programm A meldet am Ende, ob die Übertragung insgesamt erfolgreich war. Alle Ergebnisse (die beiden pcap-Dateien, die von B gespeicherte Textdatei und die Logs) legen die Programme gesammelt in einem Ordner `output/` ab.

7. Testaufbau

Für die Erprobung reicht ein einziger Rechner: Programm A und Programm B laufen in zwei getrennten Prozessen und reden ausschließlich über die Loopback-Schnittstelle miteinander. Die Daten gehen also trotzdem echt durch den Netzwerkstack und nicht am Protokoll vorbei. Zuerst wird B gestartet, es wartet dann auf eingehende Pakete; danach startet A und beginnt zu senden. Für zwei getrennte Maschinen oder virtuelle Gäste im selben Subnetz ändert sich nur die Zieladresse und die Netzwerkschnittstelle, der Rest bleibt gleich. Weil beide Programme mit Rohpaketen arbeiten, brauchen sie Administratorrechte; unter macOS und Linux werden sie deshalb mit `sudo` gestartet.

Dass am Ende wirklich dasselbe herauskommt, lässt sich mit einem Handgriff bestätigen: Ein Vergleich der Originaldatei mit der von B geschriebenen Datei darf keinen Unterschied zeigen. Zusammen mit dem SHA-256-Abgleich im Programm selbst ist das der Nachweis, dass die Übertragung zeichengenau war.

8. Ein Durchlauf in der Praxis

Das folgende Logbeispiel zeigt den typischen Ablauf einer Übertragung: A sendet der Reihe nach INIT, die 61 Datenpakete und abschließend FIN. Für jedes Paket erscheint eine Zeile mit Pakettyp, laufender Nummer und der Versuchszahl, dazu die Antwort, die der Sniffer-Thread parallel mitschreibt, etwa so:

```
-> ptype=2 seq=12 (Versuch 1/5)
<- ACK seq=12
```

Auf der Gegenseite protokolliert B jedes empfangene Paket. Sobald FIN eintrifft, setzt B die Stücke zusammen, prüft den SHA-256, speichert die Datei und gibt den vollständigen Gesetzestext auf dem Bildschirm aus. Geht einmal ein Paket verloren, sieht man im Log von A genau, welche Nummer mit welchem Versuch erneut geschickt wurde, und bei B taucht vorher eine Zeile mit CRC-Fehler und NACK auf, falls das Paket beschädigt statt verloren war.

9. Fazit

In einem kleinen, störungsarmen Testnetz arbeitet die Übertragung über ICMP zuverlässig. Die Datei kommt vollständig und zeichengenau an, und das ARQ-Verfahren gleicht einzelne verlorene oder beschädigte Pakete automatisch aus, ohne dass manuell eingegriffen werden muss. Für den Rahmen dieser Prüfungsleistung ist die Methode damit gut geeignet.

In der Praxis hat der Ansatz jedoch Grenzen. Das Stop-and-Wait-Verfahren ist bewusst einfach gehalten und nicht auf Geschwindigkeit optimiert. Für wenige Kilobyte fällt dies nicht ins Gewicht, bei größeren Datenmengen würde die Übertragung jedoch deutlich länger dauern. Obwohl Ping-Verkehr auf den ersten Blick unauffällig wirkt, lässt er sich bei genauerer Analyse erkennen: Eine hohe Anzahl von Echo-Requests mit stets gleicher Größe und einer Nutzlast mit hoher Entropie stellt für ein Überwachungssystem ein erkennbares Warnsignal dar.

An diesem Punkt setzt die Abwehr an. Ein solcher Datenabfluss lässt sich erkennen, indem der ICMP-Verkehr überwacht statt ungefiltert zugelassen wird: durch Beobachtung ungewöhnlich häufiger oder regelmäßiger Pings, Inspektion der Nutzlast auf hohe Zufälligkeit (Entropie) sowie Meldung entsprechender Muster über ein IDS wie Suricata. Verhindern lässt er sich, indem ICMP an den Netzgrenzen eingeschränkt oder in der Rate begrenzt und über Egress-Filter kontrolliert wird, welcher Verkehr das Netz verlassen darf.

Die hierfür erforderlichen Maßnahmen lassen sich in drei Bereiche gliedern. Erstens **Sichtbarkeit**: Erforderlich ist ein Sensor an der richtigen Stelle (typischerweise am Gateway oder über einen Mirror-Port am Switch), der den ICMP-Verkehr erfasst, sowie Werkzeuge, die nicht nur Pakettypen zählen, sondern auch die Nutzlast inspizieren und über NetFlow oder Monitoring Mengen und Zeitabstände über einen längeren Zeitraum auswerten. Zweitens **Regeln und Technik**: eine Firewall mit Egress-Filterung, IDS/IPS-Signaturen, die auf auffällige Payload-Größen und hohe Entropie ansprechen, und eine Rate-Begrenzung für ICMP. Drittens **Menschen und Prozesse**: Eine Meldung ist nur wirksam, wenn sie auch ausgewertet wird; dafür ist Personal erforderlich, das die Baseline des normalen Ping-Verkehrs kennt, Abweichungen bewertet und im Ernstfall reagiert. Technische Maßnahmen allein genügen daher nicht; Erkennung und Abwehr setzen sowohl die Überwachung des Verkehrs als auch eine zuständige Person voraus.